Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

第六章 Flask-模板 回顾

- 模板　　MTV、模板
- Jinja2　　简介、渲染、变量、过滤器、控制结构、继承、集成Bootstrap、include和宏
- 链接和静态文件　　链接、静态文件

第七章 Flask-Web表单

- Flask-WTF
- 基本实现

第七章 Flask-Web表单

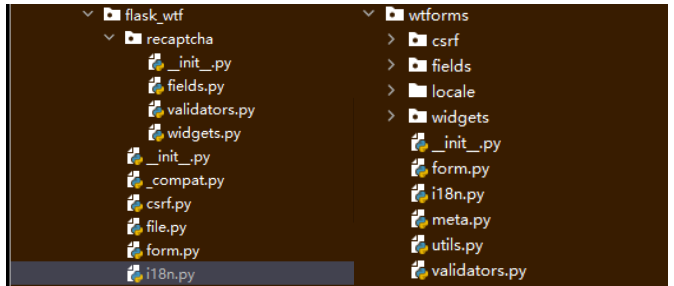
- Flask-WTF
- 基本实现

Flask-WTF

https://flask-wtf.readthedocs.io/en/1.2.x/

https://wtforms.readthedocs.io/en/3.0.x/

- 表单及其验证的必要性
  - 实现用户向服务器端提交数据
  - 解决用户输入的正确性判定，防止提交不符合要求的数据
- Flask-WTF
  - WTForms是为python Web开发服务的一个灵活的表单验证、渲染库，能够用于多种框架和模板引擎，支持数据验证，CSRF(跨站请求伪造)保护。
  - Flask-WTF对WTForms进行了封装，方便集成到Flask应用中
  - Flask-WTF可实现在后端定义表单项，利用参数传递到模板中，来实现表单创建
  - 安装：pip install flask-wtf



https://zhuanlan.zhihu.com/p/366715378

第七章 Flask-Web表单

- Flask-WTF
- 基本实现

基本实现

- 实现一个基于Flask-WTF的Web表单的基本流程
  - 配置
  - 构建表单类
  - 创建表单实例
  - 表单渲染成HTML
  - 提交验证

基本实现

配置

- Flask-WTF不用在应用层创建实例，只需要配置一个密钥
- 密钥就是一个随机的字符串，每个应用因不同且需保密，Flask使用这个密钥保护用户会话
- app.config[‘SECRECT_KEY’] = 随机的字符串，也可以使用setting.py文件来设置
- Flask-WTF为所有的表单生成安全令牌，存储在用户会话中，防止表单遭到跨站伪造请求(CSFC)攻击

https://zhuanlan.zhihu.com/p/366715378

基本实现

构建表单类

- 使用Flask-WTF，服务器端每个Web表单都由一个继承自FlaskForm的类表示
- 在类中定义表单的一组字段，每个字段都由对象表示；字段对象可附属一个或多个验证函数（用于验证用户提交的数据是否有效）

```
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Length

class UserForm(FlaskForm):
    name = StringField('Name', validators=[DataRequired(),Length(min=1,max=12)])
    password = PasswordField('Password', validators=[DataRequired(),Length(min=1,max=12)])
    submit = SubmitField('Submit')
```

基本实现

- WTForms支持的HTML标准字段如下表 \envs\flaskDemo\Lib\site-packages\wtforms\fields\simple.py

字段类型	说明	字段类型	说明
StringField	文本字段	BooleanField	复选框
TextAreaField	多行文本字段	RadioField	一组单选框
PasswordField	密码文本字段	SelectField	下拉列表
SelectMultipleField	下拉列表，可选择多个值	FileField	文件上传字段
DateField	文本字段，值为datetime.date格式	MultipleFileField	多文件上传字段
DateTimeField	文本字段，值为datetime.datetime 格式	SubmitField	表单提交按钮
IntegerField	文本字段，值为整数	FormField	把表单作为字段嵌入另一个表单
DecimalField	文本字段，值为decimal.Decimal	FieldList	一组指定类型的字段
FloatField	文本字段，值为浮点数	HiddenField	隐藏文本字段

基本实现

- WTForms内建的验证函数如下表

验证函数	说明	验证函数	说明
DataRequired	确保转换类型后字段中有数据	NumberRange	验证输入的值在数字范围内
Email	验证电子邮件地址	Optional	无输入值时跳过其他验证函数
EqualTo	比较两个字段的值；常用于要求输入两次密码进行确认的情况	Regexp	使用正则表达式验证输入值
InputRequired	确保转换类型前字段中有数据	URL	验证URL
IPAddress	验证IPv4 网络地址	AnyOf	确保输入值在可选值列表中
Length	验证输入字符串的长度	NoneOf	确保输入值不在可选值列表中
MacAddress	验证MAC地址	UUID	验证UUID



基本实现

创建表单实例

- 一般在视图函数中创建表单实例，并将实例作为参数传递给模板供模板渲染

```
from flask import render_template

@app.route('/',methods=['GET','POST'])
def index():
    uform = UserForm()
    return render_template('user.html', uform=uform)
```

基本实现

表单渲染成HTML-第一种方式

- 表单字段是可调用的，在模板中调用后就会渲染成HTML
- 调用字段是传入的任何关键字参数都将转换成字段的HTML属性，如id或class，据此方便定义其CSS

```
<form action="/" method="POST">
    <p>{{uform.csrf_token}}</p>
    <p>{{uform.name.label}}:{{uform.name}}</p>
    <p>{{uform.password.label}}:{{uform.password(id="myPassWord")}}</p>
    <p>{{uform.submit}}</p>
</form>
```

基本实现

表单渲染成HTML-第二种方式

- 使用第一种方式能够指定HTML属性，但其渲染和美化表单工作量大，因此可以使用Bootstrap的表单样式
- Bootstrap提供quick_form(表单对象)快速渲染和form_field(字段对象)逐个渲染

```
{% extends 'bootstrapBase.html' %}
{% import "bootstrap/wtf.html" as wtf %}
{% block title %}测试bootstrap创建表单{% endblock %}
{% block page_content %}
  <div class="page-header">
    <h1>Hello,{% if name %}{{ name }} {% else %}
        Stranger {% endif %}</h1>
  </div>
  {{wtf.quick_form(uform)}}
{% endblock %}
```

```
<form class="form" method="POST">
    {{ uform.csrf_token }}
    {{ wtf.form_field(uform.name) }}
    {{ wtf.form_field(uform.password) }}
    {{ wtf.form_field(uform.submit) }}
</form>
```

flaskDemo\Lib\site-packages\flask_bootstrap\templates\bootstrap\wtf.html -

基本实现

提交验证

- Flask-WTF使用 form.validate_on_submit实现表单的验证

```
@app.route('/userBootstrap', methods=['GET', 'POST'])
def index1():
    name = None
    uform = UserForm()
    if uform.validate_on_submit():
        name = uform.name.data
        uform.name.data = ""
    return render_template('userBootstrap.html', uform=uform,name=name)
```

基本实现

提交验证

- Flask-WTF也可以自定义除内建验证函数的验证函数，
- 将其定义在表单类中，其中函数名称必须是validate_<fieldname>(self,data)：
- 形参data为所验证表单的字段对象
- 自定义验证函数的执行在内建函数之后

```
def validate_name(self, data):
    if data.data[0].isdigit():
        raise ValidationError("用户名不能以数字开头")
```

基本实现

提交验证

- 也可以自定义正则验证函数，比如手机号码的验证

```
def validate_phone(self,data):
        phone = data.data
        if not re.match(r'^1[3-9]\d{9}$', phone):
            raise ValidationError(“手机号输入有误,请输入11位数字")
```

基本实现

提交验证

- 请求完成后，有时需要让用户知道状态发生了变化，可以是确认消息，警告或者错误提醒等。可以使用Flask内置的flash函数实现

flash(message: str, category: str = "message")

```
:param message: the message to be flashed.
:param category: the category for the message.  The following values
                 are recommended: ``'message'`` for any kind of message,
                 ``'error'`` for errors, ``'info'`` for information
                 messages and ``'warning'`` for warnings.  However any
                 kind of string can be used as category.
```

- 实现步骤：
  - ✓ 首先在试图函数中利用flash定义闪现的内容和类型
  - ✓ 在模板中利用循环将所有的闪现信息用get_flashed_messages()获取，并显示出来

基本实现

- Flask helpers.py 中的内置函数
  - make_response()
  - url_for()
  - redirect()
  - abort()
  - get_template_attribute()
  - flash()
  - get_flashed_messages()
  - send_file()
  - send_from_directory()
  - get_root_path()

\flaskDemo\Lib\site-packages\flask\helpers.py

第七章 Flask-Web表单 回顾

- Flask-WTF　　表单、特点
- 基本流程　　配置、构建表单类、创建实例、渲染HTML、提交验证